Recommender Systems - Course Project Report

Cumulative report template - Parts 1, 2, and 3

Team	19
Members	Kieu Duc Thinh – 152167613 Le Hoang Minh Duc – 152251833 Le Bui Minh Khoa - 152281858
For Part 3 track: select one	A / B / C
GitHub repository link	https://github.com/thinhkieu725/recommender-system

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

The dataset has 6040 users, 3883 movies, 1000209 ratings without any missing values or duplicate rows in the three data files. Interaction density is about 4.2647%, meaning that 95.7353% of possible user-movie interactions are unobserved.

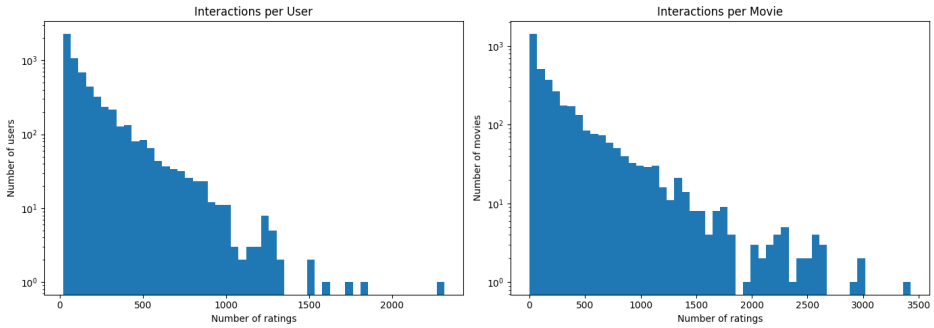


Figure 1.1 Interactions per User and Interactions per Movies

Users have an average of 165.60 ratings, while rated movies receive an average of 269.89 ratings. Both distributions are right-skewed, showing that interaction activity is unevenly distributed, with a relatively small number of users and movies accounting for a large share of interactions; notably, 177 catalogue movies have no recorded ratings.

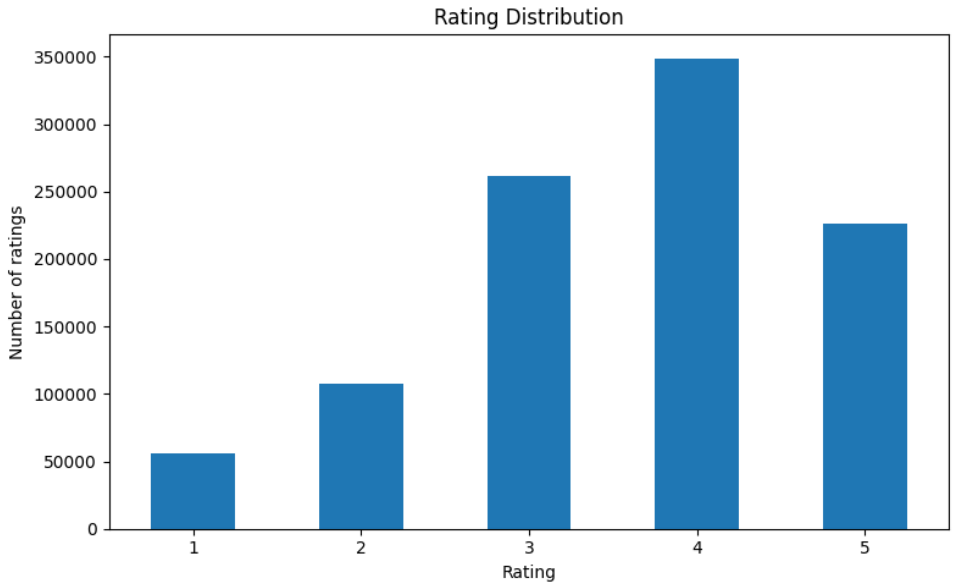


Figure 1.2 Rating distribution

Ratings range from 1 to 5, with an average of 3.58; rating 4 is the most frequent (34.89%). Ratings of 4 or 5 account for 57.52% of all interactions.

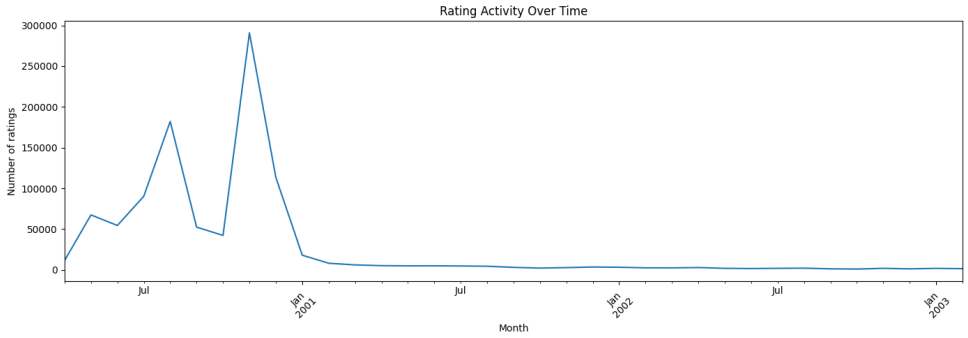


Figure 1.3 Rating activity over time

Rating activity was highest during the second half of 2000, particularly around November, before dropping sharply towards the end of the year. Activity remained much lower throughout 2001 and 2002.

1.2 Preprocessing and feedback treatment

For the baselines implemented in part 1, we chose not to conduct any filtering on the original ratings data. Twos options have been considered: filtering out low-activity users and filtering out movies with zero or very few ratings. Neither is implemented, as we believe that they do not contribute significantly to the result of the baselines. Low activity users add very few co-ratings, so they add very few noise to item-item similarities, hence little effect on item-based kNN. They only make it harder to achieve a good metric due to cold-start, which is a universal issue for all methods. Movies with very few ratings do not affect the two baselines implemented: the most popular method never touch them, and their contribution to the kNN outputs is guarded by the min_support parameter (explained in 1.5). This decision on no filtering is made based on the implementation of the current two baselines in part 1, and may change when we proceed with more methods in the next parts if necessary.

The data (downloaded as *.dat files) are loaded as pandas DataFrames. For the baselines methods in part 1, only the ratings.dat data is necessary for the two baselines, which is loaded as four columns: user ID, movie ID, ratings, and timestamps.

1.3 Train/validation/test split

We divided the ratings data temporally into training, validation, and test splits, with the corresponding ratios of 80/10/10. This ratio is chosen according to the norm in machine learning practices. We considered also other frequently used splits, such as 70/20/10 and 70/15/15, but as the train split shrinks, the proportion of users appearing in the test set but not in the training set rises significantly. This gives a harder task for most methods in general (which is acceptable), and cause the kNN to concluding more unknown answers, resulting in more noisy metrics (if calculated on the concluded entries only). Therefore, the final decision on splitting ration is 80/10/10.

The split is made temporal, meaning that all ratings are sorted by timestamp and the dataset is cut at two global points in time rather than per user. Ratings before the 80th percentile timestamp (2 December 2000) form the training set. Ratings between the 80th and 90th percentile (2 to 29 December 2000) form the validation set. The remaining ratings, up to 28 February 2003, form the test set. As in the lecture, temporal splitting reflects the real-world scenario of predicting future preferences from past interactions, avoiding the future-data leakage associated with random splitting.

1.4 Evaluation protocol

- For each user, the candidate set consists of all movies in the catalogue that they have not rated in the training and validation data. Each recommender ranks these candidates by its recommendation score, and the 10 highest-ranked movies are selected. We use full-catalogue ranking rather than sampled ranking, as the lecture explains that sampled evaluation can produce misleading comparisons between recommenders.

- Relevance definitions: for Precision@10 and Recall@10, a movie is considered relevant if its held-out rating is at least 4. For NDCG@10, the original ratings are converted to graded relevance values from 0 to 4, allowing higher-rated movies to contribute more to the score.

- As required, we have 4 metrics:

- Precision@10: Measures the proportion of the 10 recommendations that are relevant. It is position-blind, meaning that the order of relevant movies does not affect the score.
- Recall@10: Measures the proportion of all relevant held-out movies that appear in the top 10 recommendations. Together, Precision and Recall show how accurate and how complete the recommendations are.
- NDCG@10: Measures ranking quality by rewarding relevant movies appearing higher in the list. Unlike Precision and Recall, it accounts for recommendation position and supports graded relevance.
- Catalogue Coverage: Measures the fraction of the entire movie catalogue that appears in recommendation lists. This provides a beyond-accuracy measure, showing how broadly the recommender uses the catalogue.

1.5 Baseline methods and settings

The item-based kNN predicts a user's preference for an unseen item from the user's own ratings of similar items. It's implementation is made using the Surprise framework's KNNWithMeans model. This is the closest variant to the one in the lectures that we can find prebuilt available. The similarity measure chosen here is Pearson, since the data here are explicit ratings. Instead of significance weighting, we use min_support, which

serves for the same purpose: a pair with fewer than 10 co-raters gets similarity 0. The neighborhood size k is tuned using the NDCG@10 over the validation set to be 21. After that, the model is refitted on the training and validation set before tested with the test set. For each test user (except for the cold-start users not having any history in the training + validation set), the model scores and then rank the movies not previously rated in training and validation.

Two most-popular baselines uses all training and validation set history: most rated baseline, which ranks movies by the total number of ratings and most positively rated, which ranks movies by the number of ratings of at least four stars. These variants measure overall popular and most-favored feedback, respectively. For each test user, both variants recommend the top ten catalogue movies not previously rated in training and validation.

1.6 Results and analysis

The item-based kNN baseline performed sub-optimally metric-wise, compared to the most popular baseline. When calculating the metrics for all users (where not conclusive answers for cold-started users counted as 0 in metrics), we see that only around 0.55 positive test movies appeared per ten recommendations. When those not conclusive answers are excluded, the test over warm users witnessed a slight increase to 0.56 positive movies per ten recommendations. We believe this poor performance is very likely attributable to the fact that **the model is trained on a different objective that what it is tested**. The model is trained to predict the ratings, i.e. it answers the question “how would this user rate this movie if he does”, and then attached with a pipeline to convert those scores to rankings. Meanwhile, the test task (retrieval) asks “among the movies that this user will rate in the future, what are the top ones”. Therefore, the kNN’s answers appear to suggest many “niche” movies that a user would never touch.

Among the popularity baselines, counting positive ratings produced better precision, recall and NDCG than counting all ratings. It retrieved approximately 2.00 positive test movies per ten recommendations, compared with 1.92 for total-count popularity. This improvement came with slightly lower catalogue coverage, suggesting on a smaller set of movies, the concentration is greater.

2. Use of AI Assistants

Thinh: I used Claude and Claude Code (Opus 5.5) to explain the details unclear in the slides and project requirement documentation, to implement some functions and parts of functions, and to debug where necessary.

Duc: I used Codex and ChatGPT to help me understand topics that I couldn't comprehend, generate some sample codes to have an intuitive idea of where to start and fix bugs.

Khoa: I used ChatGPT to improve my code quality and to explain the concepts in the lecture that were unfamiliar to me.